

AI505
Optimization

Discrete Optimization
Randomized Optimization Heuristics

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

Outline

1. Randomized Optimization Heuristics

2. Search Methods

3. Constructive Search

Incomplete Search Ideas

Construction-Based Metaheuristics

1. Randomized Optimization Heuristics

2. Search Methods

3. Constructive Search

Incomplete Search Ideas

Construction-Based Metaheuristics

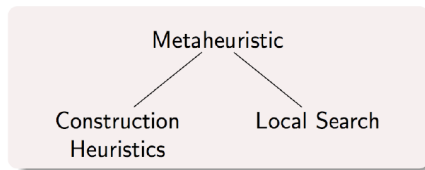
- Get inspired by approach to problem solving in human mind
[A. Newell and H.A. Simon. "Computer science as empirical inquiry: symbols and search." Communications of the ACM, ACM, 1976, 19(3)]
 - effective rules without theoretical support
 - trial and error
- Applications:
 - Optimization
 - But also in Psychology, Economics, Management [Tversky, A.; Kahneman, D. (1974). "Judgment under uncertainty: Heuristics and biases". Science 185]
- Basis on empirical evidence rather than mathematical logic. Getting things done in the given time.

Randomized Optimization Heuristics (ROHs)

Two main search paradigms:

- Constructive search (works on partial solutions)
- Local search (works on complete solutions)

plus high level guiding heuristics (ie, metaheuristics), eg, evolutionary algorithms.



Toy Examples

We will use the following examples to illustrate concepts:

- Knapsack Problem
- Traveling Salesman Problem
- Single Machine Total Weighted Tardiness
- Graph Vertex-Coloring

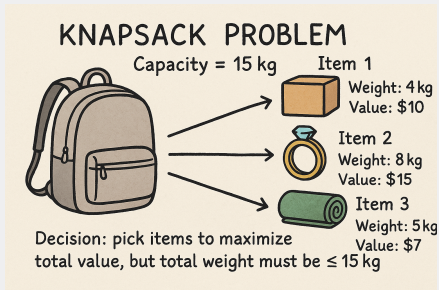
Knapsack Problem

Given: A ground set of items with weights and values. A knapsack with a weight capacity.

Task: Find the subset that maximizes the total value

Example (Instance)

Capacity = 15		
Item	Weight	Value
1	4	10
2	8	15
3	5	7
4	5	8
5	9	10
6	7	7
7	3	4
8	1	3



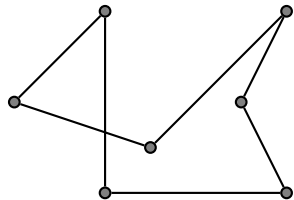
Sol. representation: Set: {1, 2, 7} or Incidence vector: [1, 1, 0, 0, 0, 0, 1, 0]

Total weight: 15; Total value: 26

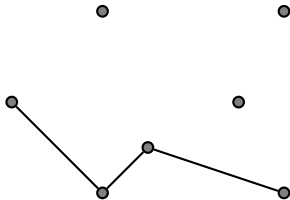
Traveling Salesman Problem

Given: A graph $G = (V, E)$ and a weight function $\omega : E \rightarrow \mathbb{R}$.

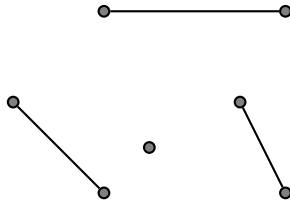
Task: Find the shortest Hamiltonian tour.



sol. representation:
permutation of vertices or
assignment of successors



sol. representation:
set of adjacent edges



sol. representation:
set of edges or
incidence binary vector

Single Machine Total Weighted Tardiness

Given: a set of n jobs $\{1, \dots, n\}$ to be processed on a single machine and for each job j a processing time p_j , a weight w_j and a due date d_j .

Task: Find a schedule that minimizes the total weighted tardiness $\sum_{j=1}^n w_j \cdot T_j$, where $T_j = \max\{C_j - d_j, 0\}$ (C_j completion time of job j)

Example (Instance)

Job	1	2	3	4	5	6
Processing Time	3	2	2	3	4	3
Due date	6	13	4	9	7	17
Weight	2	3	1	5	1	2

Sol. representation:

Linear permutation

$$\phi = [3, 1, 5, 4, 1, 6]$$

Job	3	1	5	4	2	6
C_j	2	5	9	12	14	17
T_j	0	0	2	3	1	0
$w_j \cdot T_j$	0	0	2	15	3	0

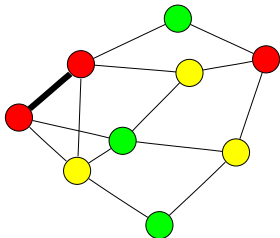
Graph Vertex-Coloring

Given: A graph G and a set of colors Γ .

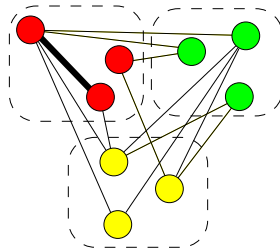
A **proper coloring**: each vertex receives a color and no two adjacent vertices receive the same color.

Task:

Find a proper coloring of G that uses the minimal number of colors (chromatic number).



Assignment representation



Partitioning representation

Outline

Randomized Optimization Heuristics

Search Methods

Constructive Search

1. Randomized Optimization Heuristics

2. Search Methods

3. Constructive Search

Incomplete Search Ideas

Construction-Based Metaheuristics

A Search Problem

Problem statement

Constrained Optimization Problem: $\min\{f(s) \mid s \in \mathcal{F}\}$

- $\mathcal{F} \subseteq \mathcal{S}$ set of **feasible solutions**
- $\mathcal{S}$ set of **candidate solutions** (**combinatorial structures**)

The concept of **feasibility** is flexible and a design choice.

Most typically, it implies satisfying the constraints of the problem.

Guiding rule: if it has an **objective function value**, then it is a **feasible solution**

Guiding rule: **constructive search algorithms** work with **partial, infeasible solutions**

Example: a partial tour does not have an objective function value (but knapsack partial solutions do have an objective function value)

Constraint Handling

Constraints in heuristic methods are handled:

- **implicitly** in the definition of the **combinatorial structures** that constitute the search states: assignments, permutations, (sub)sets, partitions, (sub)graphs, sequences, set of sequences, ...
- as **one-way** constraints between variables
- as **soft constraints**
ie, relaxed in the evaluation function as penalty components with large weights or as lexicographically more important components

A Classification

- White box optimization:
models can be expressed mathematically
- Grey box optimization:
internal information about objective function computation is often available
models that have a mathematical expression but may need data to determine them (eg, neural networks)
- Black box optimization:
no mathematical expression is available
We assume a zeroth order oracle that takes as input a point $s \in \mathcal{S}$ and outputs a dual bound if infeasible or the value of f at s .

Approaches to ROHs

- White/Grey box: representation (modelling) + reasoning (search)
constraint based local search, comet, local solver (Hexaly)
- Black Box: a different approach, framework separating problem from solvers and defining the interface specification

⇒ COST Action: ROAR-NET



<https://roar-net.eu>

A Search Problem

Definition (Search or Optimization Algorithm)

Goal formulation: we want to find the minimum with respect to some criterion from a set of candidate elements.

Problem formulation: Given a description of the states, an initial state and actions necessary to reach the goal, find a sequence of actions to reach the goal.

Search: the algorithm simulates sequences of actions in the model of the goal, searching until it finds a sequence of actions that reaches the goal.

The algorithm might have to simulate multiple tentative answers that do not meet the goal, but eventually it reaches a solution, or it will find that no solution is possible.

Components of a Search Algorithm (1)

(valid for complete or heuristic and constructive or perturbative algorithms)

- **State or (Candidate) solution**: a definition of the states of the search
- **Search Space**: A set of possible **states** that the search can be in.
- **Initial State**: State the search starts in.
- **Goal**: A set of one or more goal states. Sometimes there is one goal state sometimes there is a small set of alternative goal states
- **Evaluation function** $f(s)$: assesses the distance from a potential goal.
(note: different from "objective", it can also include penalties due to constraint violations).

Components of a Search Algorithm (2)

- **Action Type** t : available to the algorithm. Neighborhood
- A finite set of **actions** of type t that can be executed in s , $\text{Actions}(t, s)$. Move
- A **transition model** that describes what each action does. $\text{Result}(s, a)$ returns the state that results from doing action $a \in \text{Actions}(t, s)$ in state s . Apply Move
- An **action-cost function**, $\text{Action-Cost}(s, a, s')$, that gives the numeric cost of applying action a in state s to reach state s' . Increment

ROAR-NET Application Programming Interface (API)

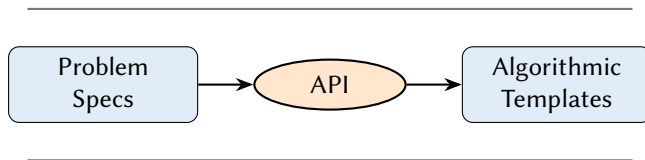
(Note: Here not meant as Web API, network-based API, or REST API.)

The ROAR-NET API Specification is the definition of an interface or protocol between optimization problems seen as black box and their solvers in order to facilitate understanding, reusing and scaling of solution approaches.

We look for a model that

- ... allows one to use off the shelf components to solve it.
- ... assumes a separation between **problem specifics** and **solver**.
- .. is designed as a **software interface** offering a service to other pieces of software and is implemented by the user.
- ... promotes **reusability** of software components and minimizes the user's effort to deploy a solution for the specific optimization problem at hand.
- ... maximizes code extensability, reusability, and simplicity.

The ROAR-NET API



- Standardized interface for **randomised optimisation algorithms**.
- Provides a set of types and methods to be **implemented by the user**, then used by the algorithms.

The ROAR-NET API

User Types

Problem

Solution

Neighborhood

Move

Value

Increment

SubNeighborhood

Methods of Problem

construction_neighbourhood

destruction_neighbourhood

local_neighbourhood

perturbation_neighbourhood

empty_solution

random_solution

heuristic_solution

Methods of Solution

lower_bound

objective_value

copy_solution

Methods of Neighborhood

moves

random_move

random_moves_without_replacement

Methods of Move

lower_bound_increment

objective_value_increment

apply_move

revert_move

<https://github.com/roar-net/roar-net-api-spec>

Minimal API Functions Implemented

Problem

```
local_neighbourhood(Problem) : Neighbourhood  
perturbation_neighbourhood(Problem) : Neighbourhood  
random_solution(Problem) : Solution
```

Solution

```
objective_value(Solution) : Value[0..1]  
copy(Solution) : Solution
```

SubNeighbourhood, Neighborhood, PerturbationNeighborhood

```
moves(Neighbourhood, Solution) : Move[0..*]  
random_move(Neighbourhood, Solution) : Move[0..1]
```

Move

```
apply_move(Move, Solution) : Solution  
revert_move(Move, Solution) : Solution  
objective_value_increment(Move, Solution) : Increment[0..1]
```

Types

- **Problem** the problem instance
- **Solution** implements the representation of a **tentative** solution
- **Value** represents points in objective space
- **Neighborhood** a function that given a solution, gives another solution $neighbor : S \rightarrow S$, based on a neighborhood, compute a **Move**
- **Move** applied to a solution to get the novel solution
- **Increment** represents difference of **Value**

Simplification in single objective cases: **Value** and **Increment** are Reals (ie, **Double** or **Integer**)

Operations: Problem Representation

... on Problem (P):

- `empty_solution(P) : Solution`
- `random_solution(P) : Solution`
- `heuristic_solution(P) : Solution[0..1]`

... on Solution S:

- `copy_solution(S) : Solution`
- `lower_bound(S) : Value[0..1]`
- `objective_value(S) : Value[0..1]`

Operations: Search

... on Neighborhood (N):

- `construction_neighbourhood(Problem): Neighbourhood`
- `destruction_neighbourhood(Problem): Neighbourhood`
- `local_neighbourhood(Problem): Neighbourhood`
- `moves(N, Solution): Move[0..*]`
- `random_move(N, Solution): Move[0..1]`
- `random_move_without_replacement(N, Solution): Move[0..*]`

... on Move:

- `lower_bound_increment(Move, Solution): double[0..1]`
- `objective_value_increment(Move, Solution): double[0..1]`
- `apply_move(M, Solution): Solution` applies the move to a solution, to get a novel solution
- `revert_move(M): Move` computes the inverse of a move to revert it

1. Randomized Optimization Heuristics

2. Search Methods

3. Constructive Search

Incomplete Search Ideas

Construction-Based Metaheuristics

Overview: Another Classification

- Solving Problems by Complete Search
 - Enumeration and Tree Search
 - Dynamic Programming
 - Backtracking (satisfaction problems)
 - Branch and Bound (optimization problems)
- Solving Problems by Incomplete Search
 - Incomplete Search Ideas (greedy algorithms)
 - Construction-Based Metaheuristics

Recall: Complete Graph Search Methods

Tree (or graph) search in

Uninformed settings

- Breadth-first search
- Uniform-cost search
- Depth-first search

Informed settings

- Greedy best-first search
- A* search

Complete Tree Search

For CSPs with n variables and d values, we can use a complete tree search with

Search Space

Tree with branching factor at the top level nd

at the next level the branching factor is $(n-1)d$.

The tree has $n! \cdot d^n$ leaves even if only d^n possible complete assignments.

- CSP is **commutative** in the order of application of any given set of action (i.e., we reach same partial solution regardless of the order)
- Hence, generate successors by considering possible assignments for only a single variable at each node in the search tree. The tree has now d^n leaves
- use information: look-ahead, best first, etc.

Exploiting Information

Assessment of partial, infeasible solutions

The priority assigned to a node x is determined by the function

$$f(x) = g(x) + h(x)$$

$g(x)$: cost of the path so far

$h(x)$: heuristic estimate of the minimal cost to reach the goal from x .

- **greedy best-first search** uses h to decide
- **A^{*} best-first search** uses f and is cost-optimal when reaches the goal, if $h(x)$ is an
 - admissible heuristic: **never overestimates** the cost to reach the goal
 - consistent: $h(n) \leq c(n, a, n') + h(n')$
(consistent $\implies$ admissible, only necessary in graph search)

Best-First Search

```
Procedure TreeSearch(start, target)  
  add start to visited  
  add start to queue  
  while queue is not empty do  
    | current_node  $\leftarrow$  vertex of queue with min distance to targeta  
    | remove current_node from queue  
    | for each neighbor n of current_node do  
    | | if n not in visited then  
    | | | if n is target then  
    | | | | return n  
    | | | else  
    | | | | mark n as visited  
    | | | | add n to queue  
  return failure
```

^a Greedy best-first: min distance to target = $h(n)$;

A* best-first: min sum of distance so far and distance to target = $f(n)$

Heuristic Functions

Possible choices for admissible heuristic functions in A^* search:

- optimal solution to an easily solvable **relaxed problem**
- optimal solution to an easily solvable **subproblem**
- learning from experience by gathering statistics on state features
- preferred heuristic functions with higher values (provided they do not overestimate)
- if several heuristics available $h_1, h_2, \dots, h_m$ and not clear which is the best then:

$$h(x) = \max\{h_1(x), \dots, h_m(x)\}$$

Drawbacks:

- Time complexity: In the worst case, the number of nodes expanded is exponential, (but it is polynomial when the heuristic function h meets the following condition:

$$|h(x) - h^*(x)| \leq O(\log h^*(x))$$

h^* is the optimal heuristic, the exact cost of getting from x to the goal.)

- Memory usage: In the worst case, it must remember an exponential number of nodes. Several variants: including iterative deepening A^* (IDA*), memory-bounded A^* (MA*) and simplified memory bounded A^* (SMA*) and recursive best-first search (RBFS)

1. Randomized Optimization Heuristics

2. Search Methods

3. Constructive Search

Incomplete Search Ideas

Construction-Based Metaheuristics

Greedy Algorithms

- Algorithms based on incomplete versions of greedy best-first search perform a single descent in the search tree.
- They always expand the current state and does not maintain an open set (frontier)
- Strategy: always make the choice that is best at the moment
- Sometimes, they guarantee the optimal solution: Minimum Spanning Tree, Single Source Shortest Path, etc.)

Greedy Algorithms in the ROAR-NET API

```
def greedy_construction(problem, solution):
    neigh = problem.construction_neighbourhood()
    if solution is None:
        solution = problem.empty_solution()
    move_iter = iter(_valid_moves_and_increments(neigh, solution))
    move_and_incr = next(move_iter, None)
    while move_and_incr is not None:
        best_move, best_incr = move_and_incr
        for move, incr in move_iter:
            if incr < best_incr:
                best_move, best_incr = move, incr
            if incr == 0:
                break
        solution = best_move.apply_move(solution)
        move_iter = iter(_valid_moves_and_increments(neigh, solution))
        move_and_incr = next(move_iter, None)
    return solution

def _valid_moves_and_increments(neigh, solution):
    for move in neigh.moves(solution):
        incr = move.objective_value_increment(solution)
        yield (move, incr) if incr < 0 else continue
```

On the backtracking framework (beyond depth-first search)

- Bounded backtrack
- Credit-based search
- Limited Discrepancy Search
- Barrier Search
- Randomization in Tree Search
- Random Restart

Outside the exact framework (beyond greedy search)

- Random Restart
- Rollout/Pilot Method
- Beam Search
- Iterated Greedy
- GRASP
- (Adaptive Iterated Construction Search)
- (Multilevel Refinement)

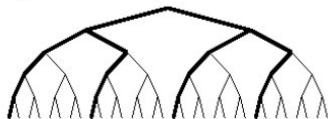
Bounded Backtrack

Bounded-backtrack search:



$bbs(10)$

Depth-bounded, then bounded-backtrack search:

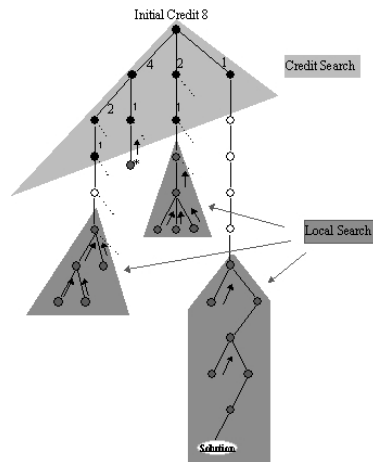


$dbb(2, bbs(0))$

Visualizations by Helmut Simonis

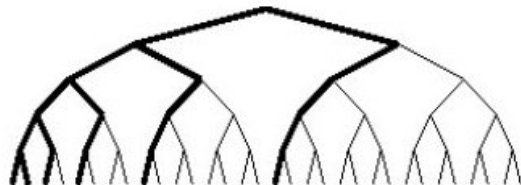
Credit-Based Search

- Key idea: important decisions are at the top of the tree
- **Credit** = backtracking steps
- Credit distribution: one half at the best child the other divided among the other children.
- When credits run out follow deterministic best-search
- In addition: allow limited backtracking steps (eg, 5) at the bottom
- **Control parameters:** initial credit, distribution of credit among the children, amount of local backtracking at bottom.



Limited Discrepancy Search

- Key observation that often the heuristic used in the search is nearly always correct with just a few exceptions.
- Explore the tree in increasing number of **discrepancies**, modifications from the heuristic choice.
- Eg: count one discrepancy if second best is chosen
count two discrepancies either if third best is chosen or twice the second best is chosen
- **Control parameter**: the number of discrepancies



Randomization in Tree Search

Idea: important decisions are made up in the search tree (backdoors - set of variables such that once they are instantiated the remaining problem simplifies to a tractable form)

~> random selections + restart strategy

Random selections

- randomization in variable ordering:
 - breaking ties at random
 - use heuristic to rank and randomly pick from small factor from the best
 - random pick among heuristics
 - random pick variable with probability depending on heuristic value
- randomization in value ordering:
 - just select random from the domain

Restart strategy with different budget in backtracking (if kept running it becomes complete)

- Example: $B = (1, 1, 2, 1, 1, 2, 4, 1, 1, 2, 1, 1, 4, 8, 1, \dots)$

Outline

Randomized Optimization Heuristics
Search Methods
Constructive Search

1. Randomized Optimization Heuristics

2. Search Methods

3. Constructive Search

Incomplete Search Ideas

Construction-Based Metaheuristics

Rollout/Pilot Method

Derived from A^*

- Master process adds components sequentially to a partial solution $S_k = (s_1, s_2, \dots, s_k)$
- At the k th iteration the master process evaluates feasible components to add based on a **heuristic look-ahead strategy**.
- The evaluation function $H(S_{k+1})$ is determined by sub-heuristics that complete the solution starting from S_k
- Sub-heuristics are combined in $H(S_{k+1})$ by
 - weighted sum
 - minimal value

Note: this evaluation does not need to be a lower bound!

Speed-ups:

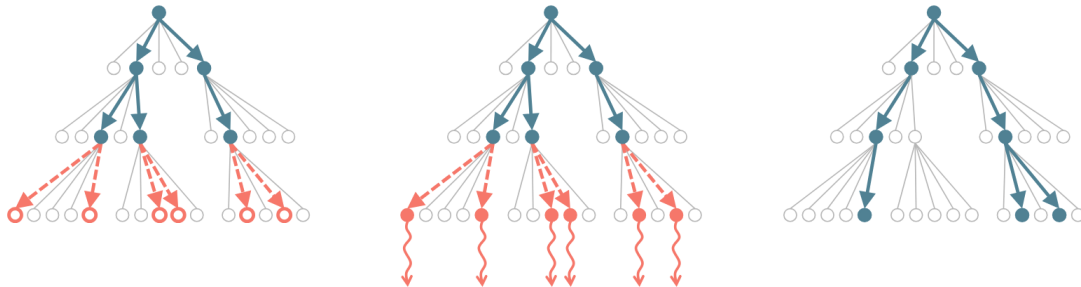
- halt whenever cost of current partial solution exceeds current upper bound
- evaluate only a fraction of possible components

Based on the tree search framework:

- maintain a set B of bw (beam width) partial candidate solutions
- at each iteration extend each solution from B in fw (filter width) possible ways
- rank each $bw \times fw$ candidate solutions and take the best bw partial solutions
- complete candidate solutions obtained by B are maintained in B_f
- stop when no partial solution in B is to be extended

Three phases of Beam Search illustrated with beam width $bw = 3$ and expansion factor $bf = 2$.

[Choo, Jinho et al. 2022]



Greedy Randomized Adaptive Search Procedure

Greedy Randomized "Adaptive" Search Procedure, aka, GRASP

Key Idea: Combine randomized constructive search with subsequent local search.

Motivation:

- Candidate solutions obtained from construction heuristics can often be substantially improved by local search.
- Local search methods often require substantially fewer steps to reach high-quality solutions when initialized using greedy constructive search rather than random picking.
- By iterating cycles of constructive + local search, further performance improvements can be achieved.

Procedure GRASP();

while **termination criterion** is not satisfied **do**

 generate candidate solution s using **subsidiary greedy randomized constructive search**;

 perform **subsidiary local search** on s ;

- Randomization in **constructive search** ensures that a large number of good starting points for **subsidiary local search** is obtained.
- Constructive search in GRASP is ‘adaptive’ (or dynamic):
Heuristic value of solution component to be added to a given partial candidate solution may depend on solution components present in it.
- Variants of GRASP without local search phase (aka **semi-greedy heuristics**) typically do not reach the performance of GRASP with local search.

- Each step of **constructive search** adds a solution component selected uniformly at random from a **restricted candidate list (RCL)**.
- RCLs are constructed in each step using a **heuristic function h** .
 - RCLs based on **cardinality restriction** comprises the k best-ranked solution components. (k is a parameter of the algorithm.)
 - RCLs based on **value restriction** comprise all solution components ℓ for which $h(\ell) \leq h_{min} + \alpha \cdot (h_{max} - h_{min})$, where h_{min} = minimal value of h and h_{max} = maximal value of h for any ℓ . (α is a parameter of the algorithm)
 - Possible extension: **reactive GRASP** (dynamic adaptation of α during search)

A GRASP Example: Squeaky Wheel

Key idea: solutions can reveal problem structure which maybe worth to exploit.

Use a greedy heuristic repeatedly by prioritizing the elements that create troubles.

Squeaky Wheel

- **Constructor:** greedy algorithm on a sequence of problem elements.
- **Analyzer:** assign a penalty to problem elements that contribute to flaws in the current solution.
- **Prioritizer:** uses the penalties to modify the previous sequence of problem elements. Elements with high penalty are moved toward the front.

Possible to include a local search phase between one iteration and the other

Iterated Greedy

Key idea: use greedy construction

- alternation of **construction** and **deconstruction** phases
- an acceptance criterion decides whether the search continues from the new or from the old solution.

Procedure Iterated Greedy;

determine initial candidate solution s ;

while termination criterion is not satisfied **do**

$r \leftarrow s$;

 (randomly or heuristically) **destruct** part of s ;

 greedily **reconstruct** the missing part of s ;

 based on **acceptance criterion**, keep s or revert to $s \leftarrow r$

[Hoos, Stützle, SLS FA, 2004]

Very Large Scale Neighborhood Algorithms

- Local search algorithms typically explore small neighborhoods of a solution. Very large scale neighborhood (VLSN) algorithms are a generalization to large neighborhoods that can be searched efficiently.
- Algorithms that proceed by iteratively destroying and repairing a solution can be seen as performing a neighborhood search in a large neighborhood as well.
- Examples of this kind are:
 - Iterated greedy
 - Large Neighborhood Search (LNS) proposed by [Shaw, 1998] use constraint programming to reconstruct a solution after destroying part of it.
 - Adaptive Large Neighborhood Search (ALNS) [Røpke and Pisinger, 2006]

Large Neighborhood Search (LNS)

input: a feasible solution x

$x^b = x$;

repeat

$x^t = r(d(x))$;

if $\text{accept}(x^t, x)$ **then**

$x = x^t$;

end if

if $c(x^t) < c(x^b)$ **then**

$x^b = x^t$;

end if

until stopping criterion is met

return x^b

$d()$ destruction (destroy) method

$r()$ reconstruction (repair) method

The LNS metaheuristic does not search the entire neighborhood of a solution, but merely samples this neighborhood.

Large Neighborhood Search (LNS)

Acceptance criterion:

- always
- only if better
- record-to-record travel
(accept if $f(s') \leq (1 + \epsilon)f(s_b)$)
- threshold accepting (Metropolis criterion)
- simulated annealing criterion

Degree of destruction

- gradually increase
- randomly chosen from a specific range
dependent on the instance size

To guarantee connectivity, it must be possible to destroy every part of the solution.

Repair method:

- problem-specific heuristic
- exact method
- general purpose mixed integer programming (MIP) (aka, fix and optimize)
- constraint programming solver (aka, fix and optimize)

It should allow diversification

Adaptive Large Neighborhood Search (ALNS)

Key Idea: allow multiple destroy and repair methods controlling with an adaptive weighting system how often a particular method is attempted during the search [Røpke, Pisinger, 2006]

input: a feasible solution x

$x^b = x$; $\rho^- = (1, \dots, 1)$; $\rho^+ = (1, \dots, 1)$;

repeat

 select destroy and repair methods $d \in \Omega^-$ and $r \in \Omega^+$ using ρ^- and ρ^+ ;

$x^t = r(d(x))$;

if $\text{accept}(x^t, x)$ **then**

$x = x^t$;

end if

if $c(x^t) < c(x^b)$ **then**

$x^b = x^t$;

end if

 update ρ^- and ρ^+ ;

until stopping criterion is met

return x^b

Adaptive Large Neighborhood Search (ALNS)

Selection mechanism: roulette wheel principle:

$$p(j) = \frac{\rho_j^-}{\sum_{k \in \Omega^-} \rho_k^-}$$

Update mechanism:

$$\Psi = \max \begin{cases} \omega_1 & \text{if the new solution is a new global best} \\ \omega_2 & \text{if the new solution is better than the current one} \\ \omega_3 & \text{if the new solution is accepted} \\ \omega_4 & \text{if the new solution is rejected} \end{cases}$$

with normally $\omega_1 \geq \omega_2 \geq \omega_3 \geq \omega_4 \geq 0$. Only accepted a and b are updated:

$$\rho_a^- = \lambda \rho_a^- + (1 - \lambda) \Psi, \quad \rho_b^+ = \lambda \rho_b^+ + (1 - \lambda) \Psi$$

$\lambda \in [0, 1]$ is a decay parameter

ALNS: Design Choices

Destroy methods:

- diversification: random destroy method.
- intensification: remove q “critical” variables, i.e. variables having a large cost or variables that spoil the current structure of the solution (e.g. edges crossing each other in an Euclidean TSP). This is known as **worst destroy** or **critical destroy**.
- **related destroy**: select a set of customers that have a high mutual relatedness measure. Eg on the CVRP, relatedness measure between each pair of customers is distance between the customers (and it could include customer demand)
- **history based destroy**: q variables are chosen according to some historical information,

Repair methods:

- greedy heuristics, problem specific
- include local search
- exact algorithms
- mixed integer programming (aka, matheuristic)
- constraint programming

ALNS: Design Choices

Large multiple-neighborhood search (LMNS) heuristics: It may be sufficient to have a number of destroy and repair heuristics that are selected randomly with equal probability, that is, without the adaptive layer.

Same robustness as ALNS heuristics, while fewer parameters to calibrate.

ALNS: Other Relations

- Variable Neighborhood Search
- Portfolio Algorithms
- Hyperheuristics
- Reinforcement learning

Adaptive Iterated Construction Search (AICS)

Key Idea: Alternate construction and local search phases as in GRASP, exploiting experience gained during the search process.

Realisation:

- Associate **weights** with possible decisions made during constructive search.
- Initialize all weights to some small value τ_0 at beginning of search process.
- After every cycle (= constructive + local search phase), update weights based on solution quality and solution components of current candidate solution.

Adaptive Iterated Construction Search (AICS)

Procedure AICS();

initialise weights;

while termination criterion is not satisfied **do**

 generate candidate solution s using subsidiary randomized constructive search;

 perform subsidiary local search on s ;

 adapt weights based on s ;

Subsidiary constructive search:

- The solution component to be added in each step of **constructive search** is based on:
 - i) **weights** and
 - ii) heuristic function h .
- h can be standard heuristic function as, e.g., used by greedy heuristics
- It is often useful to design solution component selection in constructive search such that any solution component may be chosen (at least with some small probability) irrespective of its weight and heuristic value.

Subsidiary local search:

- As in GRASP, local search phase is typically important for achieving good performance.
- Can be based on Iterative Improvement or more advanced LS method (the latter often results in better performance).
- Tradeoff between computation time used in construction phase **vs** local search phase (typically optimized empirically, depends on problem domain).

Weight updating mechanism:

- Typical mechanism: increase weights of all solution components contained in candidate solution obtained from local search.
- Can also use aspects of search history;
e.g., **current candidate solution** can be used as basis for weight update for additional intensification.

Example: A simple AICS algorithm for the TSP (1/2)

[Based on Ant System for the TSP, Dorigo et al. 1991]

- Search space and solution set as usual (all Hamiltonian cycles in given graph G). However represented in a construction tree T .
- Associate weight τ_{ij} with each edge (i, j) in G and T
- Use heuristic values $\eta_{ij} := 1/w_{ij}$.
- Initialize all weights to a small value τ_0 (parameter).
- **Constructive search** start with randomly chosen vertex and iteratively extend partial round trip π by selecting vertex not contained in π with probability

$$\Pr(\pi_j) = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in N'(i)} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta}$$

Example: A simple AICS algorithm for the TSP (2/2)

- **Subsidiary local search** = typical iterative improvement
- **Weight update** according to

$$\tau_{ij} := (1 - \rho) \cdot \tau_{ij} + \Delta(ij, s')$$

where $\Delta(i, j, s') := 1/f(s')$, if edge ij is contained in the cycle represented by s' , and 0 otherwise.

- Criterion for weight increase is based on intuition that edges contained in short round trips should be preferably used in subsequent constructions.
- Decay mechanism (controlled by parameter ρ) helps to avoid unlimited growth of weights and lets algorithm forget past experience reflected in weights.
- (Just add a population of cand. solutions and you have an Ant Colony Optimization Algorithm!)